开发指南

官方 Java SDK

Copy page

Z.ai Java SDK 是智谱AI 官方提供的 Java 开发工具包，专为与智谱AI 的各种人工智能模型进行交互而设计，为 Java 开发者提供便捷、高效的 AI 模型集成方案。

最新 Java SDK 版本为 `0.3.3`，请及时更新以获取最新功能和修复。

核心优势

企业级
专为企业应用设计，支持高并发、高可用性

易于集成
简洁的 API 设计，完善的文档，快速集成到现有项目

类型安全
完整的类型定义，编译时错误检查，减少运行时错误

高性能
优化的网络请求处理，支持连接池和异步调用

支持的功能

- **对话聊天**：支持单轮和多轮对话，流式和非流式响应
- **函数调用**：让 AI 模型调用您的自定义函数

- **视觉理解**：图像分析、视觉问答
- **图像生成**：根据文本描述生成高质量图像
- **视频生成**：文本到视频的创意内容生成
- **语音处理**：语音转文字、文字转语音
- **文本嵌入**：文本向量化，支持语义搜索
- **智能助手**：构建专业的 AI 助手应用

技术规格

环境要求

- **Java 版本**：Java 1.8 或更高版本
- **构建工具**：Maven 3.6+ 或 Gradle 6.0+
- **网络要求**：支持 HTTPS 连接
- **API 密钥**：需要有效的智谱AI API 密钥

依赖管理

SDK 采用模块化设计，您可以根据需要选择性引入功能模块：

- **核心模块**：基础 API 调用功能
- **异步模块**：异步和并发处理支持
- **工具模块**：实用工具和辅助功能

快速开始

环境要求

Java 版本

构建工具

Java 1.8 或更高版本

Maven 3.6+ 或 Gradle 6.0+

支持 Java 8, 11, 17, 21 版本，跨平台兼容 Windows、macOS、Linux

添加依赖

Maven Gradle

```
<dependency>
    <groupId>ai.z.openapi</groupId>
    <artifactId>zai-sdk</artifactId>
    <version>0.3.3</version>
</dependency>
```

获取 API Key

1. 访问 **Z 智谱开放平台**
2. 注册并登录您的账户
3. 在 **API Keys** 管理页面创建 API Key
4. 复制您的 API Key 以供使用

建议将 API Key 设置为环境变量：`export ZAI_API_KEY=your-api-key` 替代硬编码到代码中，以提高安全性。

智谱AI 国内平台使用 ZhipuAiClient 客户端

```
API 地址: https://open.bigmodel.cn/api/paas/v4/
```

创建客户端

BigModel

环境变量　直接设置

```java
import ai.z.openapi.ZhipuAiClient;

public class QuickStart {
    public static void main(String[] args) {
        // 从环境变量读取 API Key
        ZhipuAiClient client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey(System.getenv("ZAI_API_KEY"))
            .build();

        // 或者直接使用（如果已设置环境变量）
        ZhipuAiClient client2 = ZhipuAiClient.builder().ofZHIPU().build();
    }
}
```

基础对话

```java
import ai.z.openapi.ZhipuAiClient;
import ai.z.openapi.service.model.*;
import ai.z.openapi.core.Constants;
import java.util.Arrays;

public class BasicChat {
    public static void main(String[] args) {
        // 初始化客户端
        ZhipuAiClient client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey("YOUR_API_KEY")
            .build();

        // 创建聊天完成请求
        ChatCompletionCreateParams request = ChatCompletionCreateParams.builder()
            .model("glm-5.1")
            .messages(Arrays.asList(
                ChatMessage.builder()
                    .role(ChatMessageRole.USER.value())
                    .content("你好，请介绍一下自己")
                    .build()
            ))
            .build();

        // 发送请求
        ChatCompletionResponse response = client.chat().createChatCompletion(reque

        // 获取回复
        if (response.isSuccess()) {
            Object reply = response.getData().getChoices().get(0).getMessage();
            System.out.println("AI 回复: " + reply);
        } else {
            System.err.println("错误: " + response.getMsg());
        }
    }
}
```

流式对话

```java
import ai.z.openapi.ZhipuAiClient;
import ai.z.openapi.service.model.*;
import ai.z.openapi.core.Constants;
import java.util.Arrays;

public class StreamingChat {
    public static void main(String[] args) {
        ZhipuAiClient client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey("YOUR_API_KEY")
            .build();

        // 创建流式聊天请求
        ChatCompletionCreateParams request = ChatCompletionCreateParams.builder()
            .model("glm-5.1")
            .messages(Arrays.asList(
                ChatMessage.builder()
                    .role(ChatMessageRole.USER.value())
                    .content("写一首关于春天的诗")
                    .build()
            ))
            .stream(true)
            .build();

        // 处理流式响应
        ChatCompletionResponse response = client.chat().createChatCompletion(reque

        if (response.isSuccess() && response.getFlowable() != null) {
            response.getFlowable().subscribe(
                data -> {
                    // 处理流式数据块
                    if (data.getChoices() != null && !data.getChoices().isEmpty()
                        Delta content = data.getChoices().get(0).getDelta();
                        System.out.print(content);
                    }
                },
                error -> System.err.println("\n 流式错误: " + error.getMessage()),
                () -> System.out.println("\n 流式完成")
            );
        }
```

完整示例

```java
import ai.z.openapi.ZhipuAiClient;
import ai.z.openapi.service.model.*;
import ai.z.openapi.core.Constants;
import java.util.*;

public class ChatBot {
    private final ZhipuAiClient client;
    private final List<ChatMessage> conversation;

    public ChatBot(String apiKey) {
        this.client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey(apiKey)
            .build();
        this.conversation = new ArrayList<>();
        // 添加系统消息
        this.conversation.add(ChatMessage.builder()
            .role(ChatMessageRole.SYSTEM.value())
            .content("你是一个友好的 AI 助手")
            .build());
    }

    public Object chat(String userInput) {
        try {
            // 添加用户消息
            conversation.add(ChatMessage.builder()
                .role(ChatMessageRole.USER.value())
                .content(userInput)
                .build());

            // 创建请求
            ChatCompletionCreateParams request = ChatCompletionCreateParams.builde
                .model("glm-5.1")
                .messages(conversation)
                .temperature(1.0f)
                .maxTokens(1000)
                .build();

            // 发送请求
            ChatCompletionResponse response = client.chat().createChatCompletion(r
```

```
            if (response.isSuccess()) {
                // 获取 AI 回复
                Object aiResponse = response.getData().getChoices().get(0).getMess

                // 添加 AI 回复到对话历史
                conversation.add(ChatMessage.builder()
                    .role(ChatMessageRole.ASSISTANT.value())
                    .content(aiResponse)
                    .build());

                return aiResponse;
            } else {
                return "发生错误: " + response.getMsg();
            }

        } catch (Exception e) {
            return "发生错误: " + e.getMessage();
        }
    }

    public static void main(String[] args) {
        ChatBot bot = new ChatBot(System.getenv("ZAI_API_KEY"));
        Scanner scanner = new Scanner(System.in);

        System.out.println("欢迎使用 Z.ai 聊天机器人! 输入 'quit' 退出。");

        while (true) {
            System.out.print("您: ");
            String input = scanner.nextLine();

            if ("quit".equalsIgnoreCase(input)) {
                break;
            }

            Object response = bot.chat(input);
            System.out.println("AI: " + response);
        }

        System.out.println("再见! ");
        scanner.close();
```

高级功能

函数调用 (Function Calling)

函数调用允许 AI 模型调用您定义的函数来获取实时信息或执行特定操作。

定义和使用函数

```java
import ai.z.openapi.ZhipuAiClient;
import ai.z.openapi.service.model.*;
import ai.z.openapi.core.Constants;
import java.util.*;

public class FunctionCallingExample {

    // 模拟天气 API
    public static Map<String, Object> getWeather(String location, String date) {
        Map<String, Object> weather = new HashMap<>();
        weather.put("location", location);
        weather.put("date", date != null ? date : "今天");
        weather.put("weather", "晴天");
        weather.put("temperature", "25°C");
        weather.put("humidity", "60%");
        return weather;
    }

    // 模拟股票 API
    public static Map<String, Object> getStockPrice(String symbol) {
        Map<String, Object> stock = new HashMap<>();
        stock.put("symbol", symbol);
        stock.put("price", 150.25);
        stock.put("change", "+2.5%");
        return stock;
    }

    public static void main(String[] args) {
        ZhipuAiClient client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey(System.getenv("ZAI_API_KEY"))
            .build();

        // 定义函数工具
        Map<String, ChatFunctionParameterProperty> properties = new HashMap<>();
        ChatFunctionParameterProperty locationProperty = ChatFunctionParameterProp
                .builder().type("string").description("City name, for example: Be
        properties.put("location", locationProperty);
        ChatFunctionParameterProperty unitProperty = ChatFunctionParameterProperty
                .builder().type("string").enums(Arrays.asList("celsius", "fahrenhe
        properties.put("unit", unitProperty);
```

```
ChatTool weatherTool = ChatTool.builder()
        .type(ChatToolType.FUNCTION.value())
        .function(ChatFunction.builder()
                .name("get_weather")
                .description("获取指定地点的天气信息")
                .parameters(ChatFunctionParameters.builder()
                        .type("object")
                        .properties(properties)
                        .required(Collections.singletonList("location"))
                        .build())
                .build())
        .build();

// 创建请求
ChatCompletionCreateParams request = ChatCompletionCreateParams.builder()
        .model("glm-5.1")
        .messages(Collections.singletonList(
                ChatMessage.builder()
                        .role(ChatMessageRole.USER.value())
                        .content("北京今天天气怎么样？")
                        .build()
        ))
        .tools(Collections.singletonList(weatherTool))
        .toolChoice("auto")
        .build();

// 发送请求
ChatCompletionResponse response = client.chat().createChatCompletion(reque

if (response.isSuccess()) {
    // 处理函数调用
    ChatMessage assistantMessage = response.getData().getChoices().get(0)
    if (assistantMessage.getToolCalls() != null && !assistantMessage.getTo
        for (ToolCalls toolCall : assistantMessage.getToolCalls()) {
            String functionName = toolCall.getFunction().getName();

            if ("get_weather".equals(functionName)) {
                Map<String, Object> result = getWeather("北京", null);
                System.out.println("天气信息: " + result);
            }
```

```
                }
            } else {
                System.out.println(assistantMessage.getContent());
            }
        } else {
            System.err.println("错误: " + response.getMsg());
        }
    }
}
```

多模态处理

图像理解

```java
import ai.z.openapi.ZhipuAiClient;
import ai.z.openapi.service.model.*;
import ai.z.openapi.core.Constants;
import java.util.*;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Base64;

public class ImageUnderstanding {
    public static void main(String[] args) throws Exception {
        ZhipuAiClient client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey(System.getenv("ZAI_API_KEY"))
            .build();

        // 方式1: 使用图像 URL
        ChatCompletionCreateParams request1 = ChatCompletionCreateParams.builder()
            .model(Constants.ModelChatGLM4V)
            .messages(Arrays.asList(
                ChatMessage.builder()
                    .role(ChatMessageRole.USER.value())
                    .content("这张图片里有什么？请详细描述。")
                    .build()
            ))
            .build();

        ChatCompletionResponse response1 = client.chat().createChatCompletion(requ
        if (response1.isSuccess()) {
            System.out.println(response1.getData().getChoices().get(0).getMessage(
        }

        // 方式2: 使用 base64 编码的图像
        byte[] imageBytes = Files.readAllBytes(Paths.get("path/to/your/image.jpg")
        String base64Image = Base64.getEncoder().encodeToString(imageBytes);

        ChatCompletionCreateParams request2 = ChatCompletionCreateParams.builder()
            .model(Constants.ModelChatGLM4V)
            .messages(Arrays.asList(
                ChatMessage.builder()
                    .role(ChatMessageRole.USER.value())
                    .content("分析这张图片中的内容")
```

```
                .build()
        )
        .build();

ChatCompletionResponse response2 = client.chat().createChatCompletion(requ
if (response2.isSuccess()) {
    System.out.println(response2.getData().getChoices().get(0).getMessage(
}
    }
}
```

图像生成

```java
import ai.z.openapi.ZhipuAiClient;
import ai.z.openapi.service.image.CreateImageRequest;
import ai.z.openapi.service.image.ImageResponse;
import ai.z.openapi.core.Constants;

public class ImageGeneration {
    public static void main(String[] args) {
        ZhipuAiClient client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey(System.getenv("ZAI_API_KEY"))
            .build();

        // 图像生成
        CreateImageRequest request = CreateImageRequest.builder()
                .model(Constants.ModelCogView3)
                .prompt("一幅美丽的山水画，中国传统风格，水墨画")
                .size("1024x1024")
                .build();

        ImageResponse response = client.images().createImage(request);

        if (response.isSuccess()) {
            String imageUrl = response.getData().getData().get(0).getUrl();
            System.out.println("生成的图像 URL: " + imageUrl);
        }
    }
}
```

文本嵌入

```java
import ai.z.openapi.ZhipuAiClient;
import ai.z.openapi.service.embedding.Embedding;
import ai.z.openapi.service.embedding.EmbeddingCreateParams;
import ai.z.openapi.service.embedding.EmbeddingResponse;
import ai.z.openapi.core.Constants;
import java.util.Arrays;

public class TextEmbedding {
    public static void main(String[] args) {
        ZhipuAiClient client = ZhipuAiClient.builder().ofZHIPU()
            .apiKey(System.getenv("ZAI_API_KEY"))
            .build();

        // 基础文本嵌入
        EmbeddingCreateParams request = EmbeddingCreateParams.builder()
                .model(Constants.ModelEmbedding2)
                .input(Arrays.asList(
                        "这是第一段文本",
                        "这是第二段文本",
                        "这是第三段文本"
                ))
                .build();

        EmbeddingResponse response = client.embeddings().createEmbeddings(request)

        if (response.isSuccess()) {
            for (int i = 0; i < response.getData().getData().size(); i++) {
                Embedding embedding = response.getData().getData().get(i);
                System.out.println("文本" + (i + 1) + "的嵌入向量维度: " + embedding.
                System.out.println("前 5 个维度的值: " + embedding.getEmbedding().su
            }
        }
    }
}
```

更多资源

BigModel

GitHub 仓库

查看源代码、提交问题、参与贡献

API 参考

查看完整的 API 文档

示例项目

浏览更多实际应用示例

最佳实践

学习 SDK 使用的最佳实践

本 SDK 基于智谱AI 最新的 API 规范开发，确保与平台功能保持同步更新。建议定期更新到最新版本以获得最佳体验。

官方 Python SDK

Claude API 兼容